Smart Camera &

Video Surveillance System

Today's Features Documentation

Object Detection (YOLO)

Web Dashboard

Siren Alert System

Date: 04 October 2026

Phase: Feature Expansion (80% Complete)

Status: Implemented & Tested

Final Year B.Tech (CSE) Major Project

Table of Contents

1. PART A - Object Detection (YOLO)
2. 2.1 What is YOLO?
3. 2.2 YOLOv4-tiny + OpenCV DNN
4. 2.3 object_detector.py - method by method
5. 2.4 Suspicious object settings
6. PART B - Web Dashboard
7. 5.1 How live video streams to a browser
8. 5.2 dashboard.py (VideoProcessor)
9. 5.3 app.py (Flask routes)
10. 5.4 HTML templates
11. 5.5 How all modules work together
12. PART C - Siren Alert System
13. 10.1 SirenPlayer class
14. 10.2 How the siren sound is generated
15. 10.3 Where the siren is wired in
16. How to Run Everything
17. Files in This Update
18. Troubleshooting

PART A - Object Detection (YOLO)

FEATURE: The camera can now detect not just faces, but also OBJECTS (people, bags, cell phones, suitcases, bottles, etc.) in the frame. Suspicious objects (backpack, handbag, suitcase, cell phone) are highlighted in RED with a warning, while normal objects are drawn in BLUE.

A.1 What is YOLO?

YOLO stands for "You Only Look Once". It is a deep learning algorithm for object detection. Unlike older methods that scanned an image piece by piece, YOLO looks at the WHOLE image once (single pass) and directly predicts: what objects are there, where they are, and how confident it is about each.

How YOLO works in 5 steps:

1. The image is divided into a grid of cells (13 x 13 for tiny).
2. Each cell predicts a few bounding boxes and a confidence score.
3. Each box is labelled from 80 known classes (person, car, backpack...).
4. NMS (Non-Maximum Suppression) removes duplicate boxes on the same object.
5. Output: a list of (label, confidence, box) for every object found.

A.2 YOLOv4-tiny + OpenCV DNN

We use a lightweight version called YOLOv4-tiny (~24MB, fast on a normal laptop CPU, no graphics card needed). We do NOT run YOLO directly - we load its model files through OpenCV's "dnn" (Deep Neural Network) module. This avoids installing a heavy deep-learning framework.

Model file	Purpose
yolov4-tiny.cfg	Network architecture (layers and sizes)
yolov4-tiny.weights	Trained weights (the learned knowledge) ~24MB
coco.names	List of 80 class names the model knows

A.3 object_detector.py - method by method

__init__(self)

Constructor. Loads the network, loads class names, and gets the output layer names.

_load_network(self)

Checks the .cfg and .weights files exist, then cv2.dnn.readNetFromDarknet() builds the network. Sets backend = OpenCV, target = CPU.

_load_classes(self)

Reads coco.names line by line into a list of 80 class names.

_get_boxes(self, outputs, width, height)

The decoder. Translates YOLO raw numbers into boxes: keeps only detections above YOLO_CONFIDENCE (0.4), converts center-points to x,y,w,h, and runs NMSBoxes to remove duplicates.

detect_objects(self, frame)

Main function. Converts the frame into a "blob" (resized 416x416, scaled 0-1), feeds it to the network, runs forward(), decodes, and returns a list of dictionaries: {label, confidence, box, is_suspicious}.

draw_objects(self, frame, detections)

Draws rectangles + labels. Suspicious = RED, normal = BLUE.

count_suspicious(self, detections)

Returns how many suspicious objects are in the current frame.

A.4 Suspicious object settings (config/settings.py)

```
YOLO_CONFIDENCE = 0.4
YOLO_THRESHOLD  = 0.4
SUSPICIOUS_OBJECTS = ["backpack", "handbag", "suitcase", "cell phone", "knife", "scissors"]
```

PART B - Web Dashboard

FEATURE: The system now runs as a web application. Open a browser, and you see the LIVE camera feed with all detection boxes drawn in real time, plus pages for detection history and registered faces.

B.1 How live video streams to a browser

A browser cannot use a webcam directly. We use MJPEG streaming (Motion JPEG):

- A background THREAD captures webcam frames continuously
- Each frame is processed (face + object detection)
- The processed frame is encoded into JPEG image bytes
- Flask sends these JPEG images one after another in an endless stream
- The browser displays them like a live video

B.2 dashboard.py (class VideoProcessor)

Runs the whole video pipeline in a background thread. Key attributes:

- frame - the latest processed frame ready to stream
- faces / objects - detections from the current frame
- thread - the background threading.Thread worker
- recognizer, object_detector, email_alert, logger, siren - the modules

start() / stop()

Starts and safely stops the background thread.

_capture_loop(self)

The loop inside the thread:

- Open the webcam
- Loop while running: read frame
- -> recognize faces, draw boxes
- -> every 3rd frame run object detection (saves CPU)
- -> draw object boxes
- -> if a known face matched and cooldown passed: save snapshot, email, log, play siren
- -> store the finished frame in self.frame

get_frame_jpeg(self)

Encodes the latest frame into JPEG bytes for the browser; returns None if no frame yet.

B.3 app.py (Flask routes)

Route	Kind	What it does
/	Page	Main dashboard: stats + live feed + current detections

/video_feed	Stream	Endless JPEG video stream (MJPEG)
/alerts	Page	Table of all detection history
/known_faces	Page	List of registered faces with photos
/snapshot/<id>	Image	Serves a known face photo
/api/stats	JSON	Statistics for programs
/api/streaming_status	JSON	Email/siren/pipeline status

The MJPEG generator (the heart of live video)

```
def generate():
    while True:
        jpeg = processor.get_frame_jpeg()
        if jpeg:
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n' + jpeg + b'\r\n')
return Response(generate(), mimetype='multipart/x-mixed-replace; boundary=frame')
```

This generator never ends. Each yield is one JPEG picture separated by a --frame marker. The browser plays them one after another = live video.

Server startup

```
if __name__ == '__main__':
    processor.start()                # start camera thread
    try:
        app.run(host='0.0.0.0', port=5000, use_reloader=False)
    finally:
        processor.stop()             # stop camera on exit
```

host="0.0.0.0" = any device on your Wi-Fi can open it at http://YOUR_IP:5000. use_reloader=False avoids opening two cameras.

B.4 HTML templates (templates/)

- base.html - shared navbar (Live Feed / Alerts / Known Faces) and all CSS
- index.html - stat boxes + <img src="/video_feed"> live video + current detection badges
- alerts.html - table of last 100 detections with MATCH/UNKNOWN badges
- known_faces.html - table of registered people with photos via /snapshot/<id>

B.5 How all modules work together

```
app.py (Flask) -> src/dashboard.py (VideoProcessor thread)
                    |-> src/face_recognizer.py (faces)
                    |-> src/object_detector.py (YOLO)
                    |-> src/email_alert.py (emails)
                    |-> src/siren.py (audio alarm)
                    |-> src/detection_logger.py (CSV logs)
app.py -> src/database.py (SQLite)
app.py -> templates/ (HTML)
app.py -> /video_feed (MJPEG stream)
```

```
Browser <-> Flask <-> VideoProcessor <-> webcam
```

PART C - Siren Alert System

FEATURE: When a registered (known) person is detected, besides the email and the snapshot, the system now also plays a POLICE-STYLE SIREN sound as an audio alarm to grab immediate attention.

C.1 siren.py (class SirenPlayer)

- __init__(): stores the sound path and creates the sound file if missing
- _generate_siren_wav(): builds the siren WAV mathematically (numpy + wave)
- play(): plays via aplay/paplay (Linux), winsound (Windows), afplay (Mac)
- play_async(): plays in a background thread so the camera video never freezes
- test(): plays the siren once to check speakers

C.2 How the siren sound is generated (no internet needed)

```
t     = time samples for 3 seconds
freq  = 500 + 450 * sin(2*pi*1.5*t)   (500..950 Hz, rises+falls)
wave  = sin(2*pi * cumsum(freq))       (the sine tone)
envelope = fade in/out to avoid clicks
save  = 44100 Hz, 16-bit WAV  ->  sounds/siren.wav
```

C.3 Where the siren is wired in

```
# settings.py
SIREN_ENABLED = True
SIREN_SOUND_PATH = sounds/siren.wav

# surveillance.py and dashboard.py
from src.siren import SirenPlayer
self.siren = SirenPlayer() if SIREN_ENABLED else None
...
when a face matches:
    if self.siren:
        self.siren.play_async()
```

C.4 Why play_async()?

A 3-second sound blocks the program for 3 seconds. If we called play() directly, the live video would freeze. play_async() starts a small background thread that plays the sound while the camera loop keeps running smoothly.

How to Run Everything

Web Dashboard (recommended for demo)

```
cd ~/Desktop/Desktop/major_project
/usr/bin/python3 app.py
# open browser -> http://localhost:5000
# from phone (same Wi-Fi): http://YOUR_IP:5000
```

Terminal Surveillance (with siren)

```
/usr/bin/python3 -m src.surveillance
```

Test the pieces

```
/usr/bin/python3 -m src.siren                    # hear the siren
/usr/bin/python3 -c "from src.object_detector import ObjectDetector; print(len(ObjectDetector().classes))"
```

Files in This Update

Folder	File	What it is
src/	object_detector.py	YOLO object detection (NEW)
src/	dashboard.py	VideoProcessor background thread (NEW)
src/	siren.py	Siren audio alarm (NEW)
root	app.py	Flask web application (NEW)
templates/	base.html	Shared page layout (NEW)
templates/	index.html	Live feed page (NEW)
templates/	alerts.html	Alerts history page (NEW)
templates/	known_faces.html	Known faces page (NEW)
sounds/	siren.wav	Generated siren sound (NEW)
models/	yolov4-tiny.cfg	YOLO config (NEW)
models/	yolov4-tiny.weights	YOLO weights ~24MB (NEW)
models/	coco.names	80 class names (NEW)
config/	settings.py	Added YOLO + dashboard + siren settings
src/	database.py	Fixed thread-safety (check_same_thread=False + lock)
src/	surveillance.py	Added siren on match

Troubleshooting

"Address already in use" when running app.py

Port 5000 is busy. Close it or change the port in app.py.

```
pkill -f "app.py"   # or change port=5000 to port=5001
```

YOLO detects nothing

Make sure an object is in view. Lower YOLO_CONFIDENCE to 0.25 in settings.py for more detections (but more false positives).

Dashboard shows black/empty video

The webcam may be busy. Close other camera apps, then restart app.py.

No sound from siren

Check speakers. Verify aplay exists (Linux): `which aplay`. Sound file auto-creates in sounds/siren.wav.

"No module named flask"

```
/usr/bin/python3 -m pip install flask
```

The Complete Pipeline (Today's Full Version)

```
WEBCAM
   |
   v
VideoProcessor thread
   |-- Haar Cascade face detection
   |-- dlib 128-dim face encoding
   |-- SQLite distance match
   |-- YOLO object detection (every 3rd frame)
   |-- draw green/red face boxes + blue/red object boxes
   |
   +-- MATCH? --> snapshot + email + SIREN + log
   |
   v
self.frame (latest)
   |
   v
Flask /video_feed (MJPEG) -----> Browser live feed
Flask /alerts             -----> detection history
Flask /known_faces        -----> registered people
Flask /api/*              -----> JSON statistics
```